

```
graph TD
    subgraph cliente_softhouse [cliente · softhouse]
        A[entrega as evidências  
SQL, PDFs ou código;  
artefatos um a um] --> B[responde o questionário  
60 itens, com id;  
pode parar e retomar]
        E[recebe a entrega  
sete pastas, SHA-256,  
procedência e BOM]
    end

    subgraph engenheiro_de_conversao [engenheiro de conversão]
        C[G0 · pré-flight  
inventário com hash;  
BLOCKED trava o código] --> D[G1-G3 estrutura  
inventário, arquitetura,  
dados]
        D --> E4[G4 piloto  
uma vertical inteira,  
com golden master]
        E4 --> F[G5 ondas  
módulo a módulo,  
telas pelo Impeccable]
    end

    subgraph qa_independente [QA independente]
        G[F-GATE  
funciona?]
        H[C-GATE  
conforme?]
        I[evidência e grafo  
o que não foi provado  
fica escrito]
    end

    subgraph aprovador [aprovador]
        J{aprova o G0?}
        K{homologa?}
    end

    B -- aplica --> C
    C --> J
    J --> E
    D --> G
    E4 --> H
    F --> I
    G -- reprovado volta para quem escreveu — o QA não conserta o que detecta --> C
    H -- reprovado volta para quem escreveu — o QA não conserta o que detecta --> E4
    I -- reprovado volta para quem escreveu — o QA não conserta o que detecta --> F
    I --> K
    K --> E
```

O diagrama ilustra o processo de entrega e aprovação de evidências, organizado em quatro swimlanes horizontais:

- cliente · softhouse**:
 - entrega as evidências**: SQL, PDFs ou código; artefatos um a um.
 - responde o questionário**: 60 itens, com id; pode parar e retomar.
 - recebe a entrega**: sete pastas, SHA-256, procedência e BOM.
- engenheiro de conversão**:
 - G0 · pré-flight**: inventário com hash; BLOCKED trava o código.
 - G1-G3 estrutura**: inventário, arquitetura, dados.
 - G4 piloto**: uma vertical inteira, com golden master.
 - G5 ondas**: módulo a módulo, telas pelo Impeccable.
- QA independente**:
 - F-GATE**: funciona?
 - C-GATE**: conforme?
 - evidência e grafo**: o que não foi provado fica escrito.
- aprovador**:
 - aprova o G0?** (decisão)
 - homologa?** (decisão)

Fluxo do processo:

- O cliente entrega as evidências e responde o questionário.
- O engenheiro aplica o questionário, iniciando o G0.
- O engenheiro avança pelos gates G1, G2 e G3.
- O engenheiro avança pelo G4.
- O engenheiro avança pelo G5.
- O QA independente verifica cada gate (F-GATE, C-GATE, evidência e grafo).
- Se qualquer gate for reprovado, o processo volta para o engenheiro de conversão para correção.
- O aprovador aprova o G0 e homologa o processo final.

Legenda:

- cliente · softhouse (azul)
- engenheiro de conversão (verde)
- QA independente (laranja)
- aprovador (vermelho)

Notas:

- reprovado volta para quem escreveu — o QA não conserta o que detecta
- aqui o trabalho PARA; nenhum gate avança sem decisão humana registrada

As quatro raias

PAPEL	O QUE FAZ	O QUE O PLUGIN GARANTE
cliente · softhouse	Entrega evidência, responde o questionário, homologa e recebe.	Não escreve código do destino. O que ele entrega vira hash no inventário do G0.
engenheiro de conversão	Converte com o plugin, gate a gate.	Escreve o produto. Com <code>WX_PAPEL</code> ausente, é o papel padrão e nada muda.
QA independente	Prova: F-GATE, C-GATE, evidência, grafo, efeito.	Com <code>WX_PAPEL=qa</code> o hook recusa que ele escreva o produto que valida — quem valida não conserta o que detecta.
aprovador	Decide o que só gente decide: liberar o G0 e homologar.	É o nome registrado no item 0.16 do questionário; o kickoff o cita.

A separação entre engenheiro e QA não é organograma: é o hook `papel_da_sessao.py` . Sem papel declarado **nada muda** — a guarda entra pedida, não imposta, porque proteção que quebra quem já usava não é proteção, é estrago.